

XiaoDuan AI: Incremental Pruning and On-Demand Recall — A Technical White Paper

Author: Nuo Yan (yiliu)
Affiliation: Independent Developer, Wuhan, China
Email: 348419142@qq.com
Date: April 2026

Abstract

This paper systematically presents a novel memory management architecture for Large Language Models—"Incremental Pruning + On-Demand Recall"—covering its conceptual origins, core technical solutions, and engineering implementation paths. This architecture maintains stable Token consumption per conversation at under 4000 through constant window and time-depth indexing mechanisms, while achieving nearly unlimited long-term memory that can be recalled on demand, ensuring long-term operational stability. This solution is independent of any specific model ecosystem and has been fully implemented in the open-source project "XiaoduanAI", verified by 210,000 users with 8 months of zero complaints.

Keywords: LLM memory management; on-demand recall; incremental pruning; AI agent; persistent memory; constant window; anchor-based retrieval

1. Background and Motivation

In the second half of 2025, the mainstream exploration of "LLM memory" in the AI Agent field focused on Context Compression solutions [1][2]. The core idea is: when conversation history exceeds the window limit, a helper model summarizes the historical information and reinjects the compressed text into the context.

This approach has fundamental flaws:

Flaw	Description
Lossy Compression	Once a summary is generated, original details are permanently lost and cannot be retrieved
Passive Trigger	Compression only activates when the window is about to overflow, lacking preventive mechanisms
Increasing Costs	Compression itself requires additional LLM calls, increasing computational overhead
Passive Model	The model does not participate in memory management, only passively receiving compressed context

The author of this paper launched the "XiaoduanAI" project in August 2025, proposing a systematic alternative to the above approach. The core insight is: a single Agent task does not need complete conversation history; the model only needs to know the task instructions for the current step; if key information is missing, the model should be able to actively and precisely recall it on demand.

This concept went through early exploration and iteration of the "Four-Layer Profiling System" (August 2025), ultimately condensing into the "Incremental Pruning + On-Demand Recall" architecture, which has been continuously optimized and improved from December 2025 to present.

2. Early Exploration: Four-Layer Profiling System (Implemented November 2025)

2.1 Design Goal

Build a persistent memory system independent of the conversation window, enabling AI to retain key information from historical conversations and actively retrieve it when needed.

2.2 Architecture Design

The Four-Layer Profiling System decomposes each conversation into four dimensions:

Layer	Content	Extraction Method
Person Profile	People appearing in the conversation, their preferences and background	Local small model keyword extraction
Event Profile	Key events and decisions in the conversation	Same as above
Location Profile	Involved files, folders, tools, and platforms	Same as above
Content Profile	Summary of key information from original conversation	Same as above

For each conversation, the local small model performs preprocessing once, extracting four-layer keywords from the user's message, pre-searching related memories, filling them into context, and passing everything to the cloud LLM for processing. After the LLM returns, while passing to the frontend, the small model silently categorizes this round's dialogue summary into four layers and incrementally writes it to the memory file.

This system was functionally operational. The LLM did receive relevant information in every conversation, and chats consistently carried the needed memories.

However, a significant problem emerged: processing speed was insufficient to provide a smooth user experience. Each conversation required small model preprocessing, content pre-search, LLM generation, and then small model summarization—the processing chain was too long, resulting in noticeable latency.

User experience fluency is a critical requirement for practical deployment.

The author decided to abandon the already-functional four-layer architecture in favor of a more efficient approach.

2.3 Complete Reconstruction: December 2025 to Present

As LLMs became increasingly capable, the author shifted the design objective from "structural perfection" to "retrieval efficiency".

Memory was transformed from a "layered filing cabinet" into a "time-indexed database". Instead of requiring the model to understand complex classifications, it only needed to process one thing: keywords and time depth.

Thus, the "true permanent memory" mechanism was established. Constant window, reverse-order indexing, anchor-based recursive retrieval. The speed problem was resolved, and the user experience became fluent.

Subsequent features such as "self-evolution", "intrinsic security", and "self-healing mechanism" were all built upon this architectural foundation, progressively enhancing the system's autonomous capabilities.

3. Core Architecture: Incremental Pruning + On-Demand Recall

3.1 Conventional Approach: Context Compression

When conversation history becomes excessively long, the conventional approach is to summarize the history and reinject the compressed text into the context window [3][4].

3.2 Xiaoduan's Approach: Index Key Points, Recall on Demand

3.2.1 Constant Window

For each conversation, the system strictly separates dialogue content, tool process, and tool results, saves them independently, and enforces character limits when passing them in the next conversation:

Type	Character Limit
Dialogue Content	3000 characters
Tool Process	2000 characters
Tool Results	2000 characters

This ensures the LLM maintains clear task focus, removes irrelevant information, and prevents context inflation and overflow. As new content is added, an equivalent amount of old content is pruned.

After major LLM providers launched monthly subscription models, the author added a complex mode, increasing dialogue content to 40,000 characters, tool process to 8,000, and tool results to 20,000 to reduce API call frequency.

3.2.2 Permanent Memory Core

The LLM is provided with prompt instructions for on-demand recall. For example, when the model outputs "Zhang San 1" (where 1 represents the first match), the system precisely returns the most recent Zhang San-related memory of 500 characters.

However, human recollection is never isolated—remembering one event inevitably brings to mind surrounding events. Therefore, the author designed a divergence anchor mechanism for the first keyword.

The system simultaneously and automatically retrieves the entries closest in time:

Example 1: When the model outputs "Zhang San 1 Travel 1", it retrieves Zhang San and the nearest travel memory, each 500 characters, combined into 1000 characters

Example 2: Repeated references "Zhang San 4 Zhang San 5" retrieve more distant Zhang San memories

Example 3: Divergent search is also supported—when Travel 1's 500 characters end with "mountain climbing", using "Zhang San 1 Mountain Climbing 1 Scenery 1" can recall broader

contextual details

This effectively creates a "memory language" that the model can use independently. The model instructs the system which memory segments to assemble, and the system executes reverse-order search with near-zero latency, returning results instantly without traversing the entire memory file.

The recall control is delegated to the model itself. An effective AI assistant must autonomously determine what to remember and what to recall.

****Design Principle****: Human memory operates through indexing rather than compression. The objective is not an AI with "unlimited context", but an AI that "knows where to recall from".

3.3 Engineering Implementation Characteristics

Characteristic	Description
Constant Token Consumption	Stable at approximately 4000 tokens per conversation, regardless of conversation history growth
Unlimited Memory Capacity	Long-term memory storage has no capacity limit; all historical information is archived without loss
Model-Active Memory Management	Model autonomously determines when to recall and how deeply, controlling memory management
Zero-Latency Recall	Retrieval operates through pre-built indices with array access, consuming no additional inference resources

3.4 Security Mechanism: Pure-Text Skill Flow

This system adopts a pure-text skill description architecture (.MD documents), where all skills are stored as natural language process descriptions and executed by the core engine through parsing. Unlike the code sandbox isolation solutions widely used in the industry [5], this architecture isolates malicious code injection risks at the execution level—skill files themselves possess no system-level execution capability.

3.5 Self-Healing Mechanism: Core File Monitoring and Automatic Recovery

The system includes a core file monitoring module. When a model silent timeout is detected, it automatically rolls back core configuration files to their pre-modification state and restarts, ensuring long-term operational stability and system resilience in unattended scenarios.

4. Demonstration of Originality and Creative Contribution

4.1 Timeline of Conceptual Origins

Time	Event
August 2025	Author launched XiaoduanAI project, began independent exploration of AI memory management solutions
November 2025	Four-Layer Profiling System implemented and deployed
December 2025	Complete reconstruction to Incremental Pruning + On-Demand Recall architecture
February 2026	XiaoduanAI v1.0.0 official release
April 2026	XiaoduanAI v1.0.1 with major local model optimization

During this period, public discussions and implementations regarding "layered memory" and "on-demand injection" in the AI community appeared after the author's independent exploration:

Event	Time
Claude launched passive memory feature	August 2025
Claude automatic memory opened to paid users	October 2025
Thoughtworks evaluated Mem0 layered memory architecture	November 2025
claude-mem released persistent semantic memory solution	December 2025
Milvus analyzed Claude Code four-layer memory architecture	April 2026
MemPalace project released	April 2026
Academic paper "Cooperative Memory Paging" published on arXiv	April 2026

4.2 Core Innovations

- 1. **Constant Window Mechanism:** Achieves constant non-growing Token consumption at the architectural level, rather than passively handling overflow through compression algorithms
- 2. **Model-Led Memory Recall:** Model actively manages memory through custom retrieval syntax, rather than passively receiving system-injected context
- 3. **Time-Depth Indexing and Anchor-Based Divergent Retrieval:** A novel memory retrieval mechanism balancing precise positioning with associated context retrieval
- 4. **Pure-Text Skill Flow Security Architecture:** Isolates code injection risks at the execution level, forming a differentiated approach from current mainstream sandbox isolation solutions
- 5. **Core File Self-Healing Mechanism:** Automatically rolls back and restarts upon detecting model silent timeout, ensuring long-term system stability
- 6. **Engineering Implementation Completeness and Stability:** Maintained zero complaints from 210,000 users over 8 months of actual use, verifying production-grade usability of the architecture

5. Conclusion

The "Incremental Pruning + On-Demand Recall" architecture presented in this paper is an independent original solution to the LLM memory management problem. This solution departs from the mainstream "context compression" paradigm, redefining AI memory from the perspective of "model actively managing memory". The accompanying pure-text skill flow security architecture and core file self-healing mechanism further ensure system security and operational stability. This solution has been fully implemented in the open-source project "XiaoduanAI" and validated through large-scale user testing, demonstrating both theoretical coherence and engineering feasibility.

References

[1] Anthropic. "Claude Memory Feature." Anthropic, August 2025.

[2] thedotmack. "claude-mem: Persistent Memory Compression System for Claude Code." GitHub, December 2025. <https://github.com/thedotmack/claude-mem>

[3] Mem0. "Layered Memory Architecture for AI Agents." Thoughtworks Technology Radar, November 2025.

[4] Milvus. "Claude Code Four-Layer Memory Architecture Analysis." Milvus Blog, April 2026.

[5] OpenClaw. "Skills Ecosystem Security Analysis." GitHub Security Advisory, 2026.

Author: Nuo Yan

Affiliation: Independent Developer, Wuhan, China

Email: 348419142@qq.com

Project Address: <https://www.modelscope.cn/datasets/yiliu666/xiaoduan>

GitHub: <https://github.com/xiaoduanRobot/yiliu666>